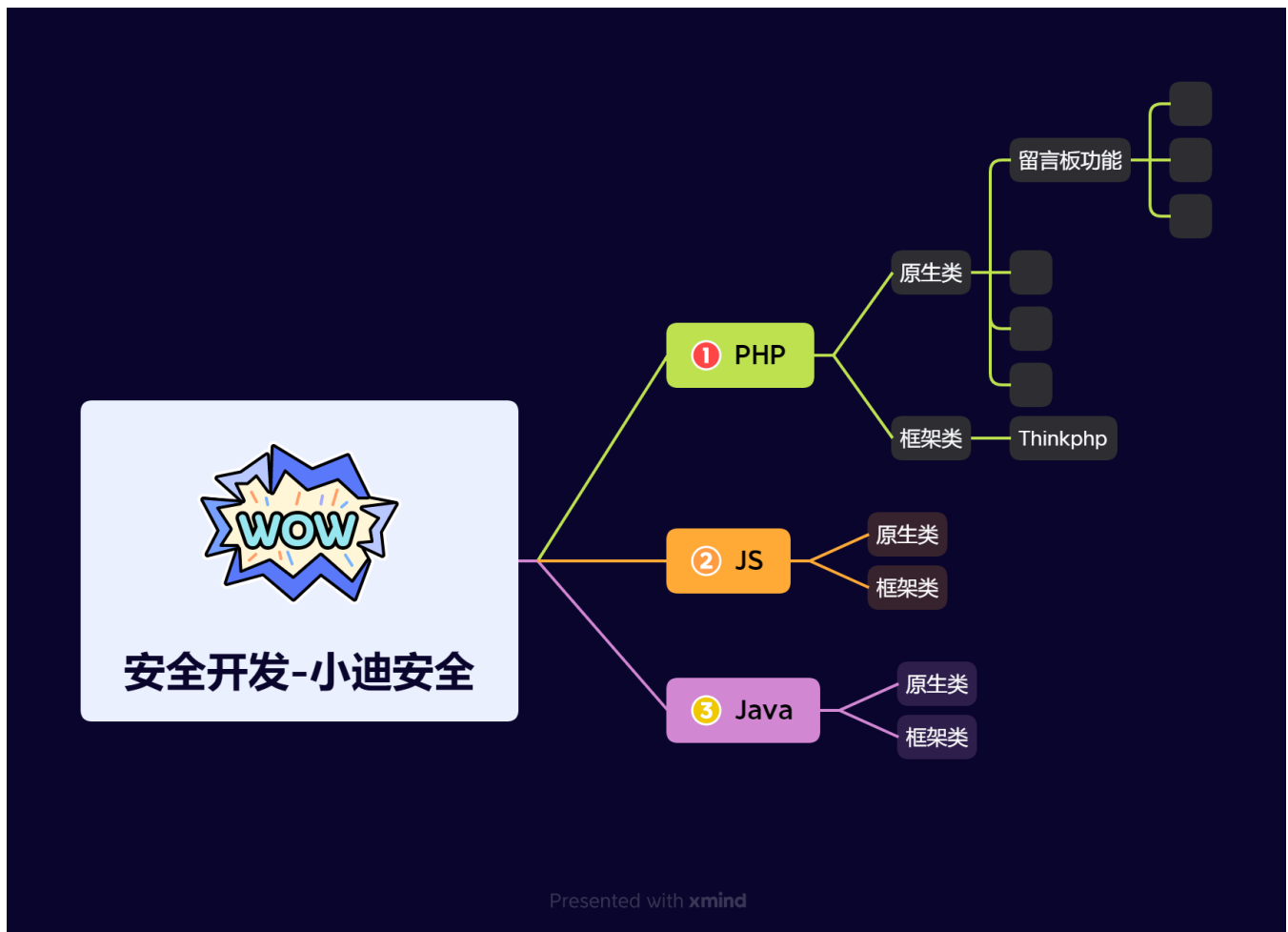


Day38 安全开发-JavaEE应用&SpringBoot框架 &MyBatis注入&Thymeleaf模版注入



1.知识点

- 1、PHP留言板前后端功能实现
- 2、数据库创建&架构&增删改查
- 3、内置超全局变量&HTML&JS混编
- 4、第三方应用插件&传参&对象调用

-
- 1、PHP后台身份验证模块实现
 - 2、Cookie&Session技术&差异
 - 3、Token数据包唯一性应用场景
 - 项目1：用cookie做后台身份验证
 - 项目2：用session做后台身份验证
 - 项目3：用token做用户登录判断
-

- 1、PHP文件管理-显示&上传功能实现
 - 2、文件上传-\$_FILES&过滤机制实现
 - 3、文件显示-目录遍历&过滤机制实现
-

- 1、PHP文件管理-下载&删除功能实现
 - 2、PHP文件管理-编辑&包含功能实现
-

- 1、PHP新闻显示-数据库操作读取显示
 - 2、PHP模版引用-自写模版&Smarty渲染
 - 3、PHP模版安全-RCE代码执行&三方漏洞
-

- 1、TP框架-开发-路由访问&数据库&文件上传&MVC模型
 - 2、TP框架-安全-不合规写法&内置过滤绕过&版本安全漏洞
-

- 1、JS应用-原生态开发&第三库开发
 - 2、JS功能-文件上传&登录验证&商品购买
-

- 1、JS技术-DOM树操作及安全隐患
 - 2、JS技术-加密编码及数据安全调试
-

- 1、NodeJS-开发环境&功能实现
 - 2、NodeJS-安全漏洞&案例分析
 - 3、NodeJS-开发指南&特有漏洞
-

- 1、三方库-JQuery-使用&安全
 - 2、打包器-Webpack-使用&安全
-

- 1、JavaEE-HTTP-Servlet技术
 - 2、JavaEE-数据库-JDBC&Mybatis
-

- 1、JavaEE-JDBC-SQL预编译
 - 2、JavaEE-HTTP-Filter过滤器
 - 3、JavaEE-对象域-Listen监听器
-

- 1、JavaEE-反射机制-开发和安全应用场景
- 2、JavaEE-反射机制-类&成员变量&方法&构造方法操作
- 3、JavaEE-反射机制-安全应用&反射执行&反序列化链相关

-
- 1、JavaEE-反序列化-解释&使用&安全
 - 2、JavaEE-安全-利用链&直接重写方法
 - 3、JavaEE-安全-利用链&外部重写方法

-
- 1、JavaEE-组件安全-Log4j
 - 2、JavaEE-组件安全-Fastjson
 - 3、JavaEE-基本了解-JNDI-API

-
- 1、JavaEE-JNDI注入-RMI&LDAP
 - 2、JavaEE-漏洞结合-FastJson链
 - 3、JavaEE-漏洞条件-JDK版本绕过

-
- 1、JavaEE-SpringBoot-WebAPP&路由
 - 2、JavaEE-SpringBoot-Mybatis&注入
 - 3、JavaEE-SpringBoot-Thymeleaf&SSTI

2.演示案例



- 1 Spring Boot是由Pivotal团队提供的一套开源框架，可以简化spring应用的创建及部署。它提供了丰富的Spring模块化支持，可以帮助开发者更轻松快捷地构建出企业级应用。Spring Boot通过自动配置功能，降低了复杂性，同时支持基于JVM的多种开源框架，可以缩短开发时间，使开发更加简单和高效。

2.1 SpringBoot-Web应用-路由响应



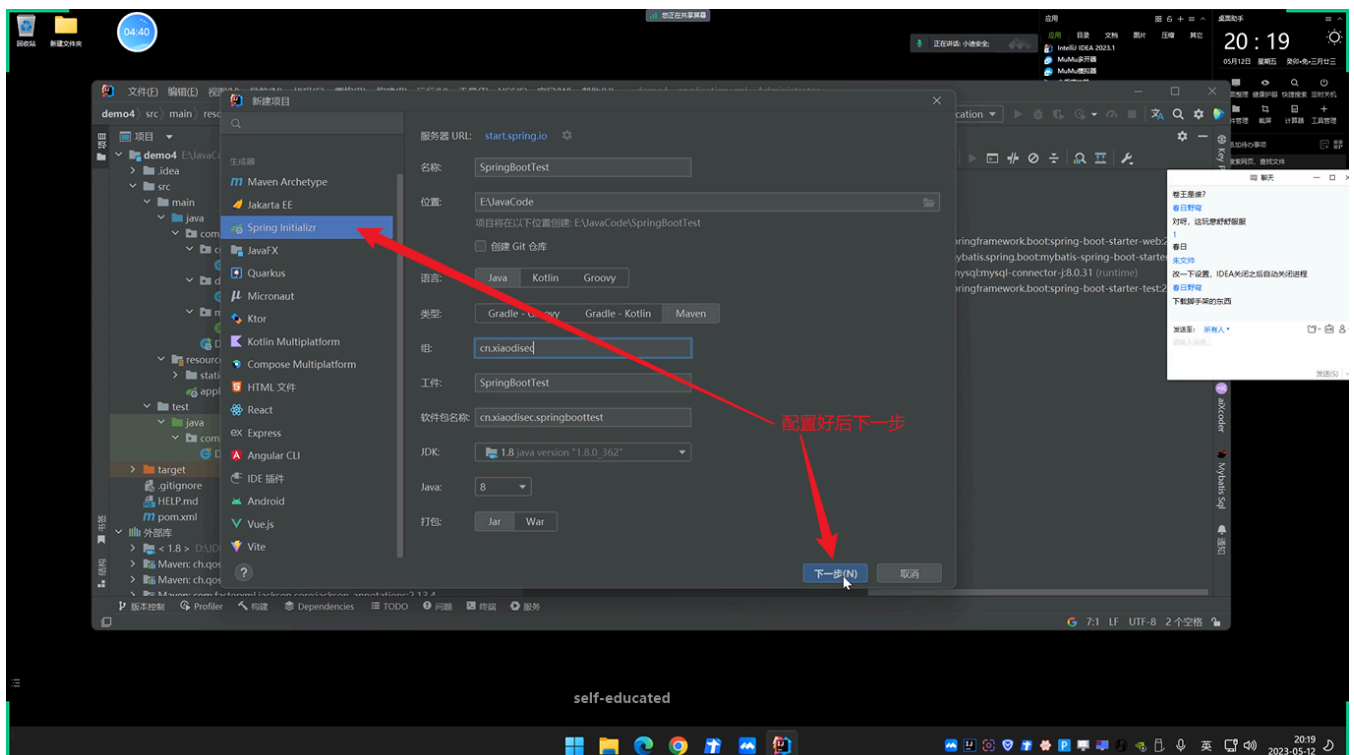
- 1 参考: <https://springdoc.cn/spring-boot/>
- 2 1、路由映射
- 3 @RequestMapping @GetMapping等
- 4 2、参数传递
- 5 @RequestParam
- 6 3、数据响应
- 7 @RestController @Controller
- 8 @RestController注解相当于@ResponseBody+@Controller合在一起的作用。
- 9
- 10

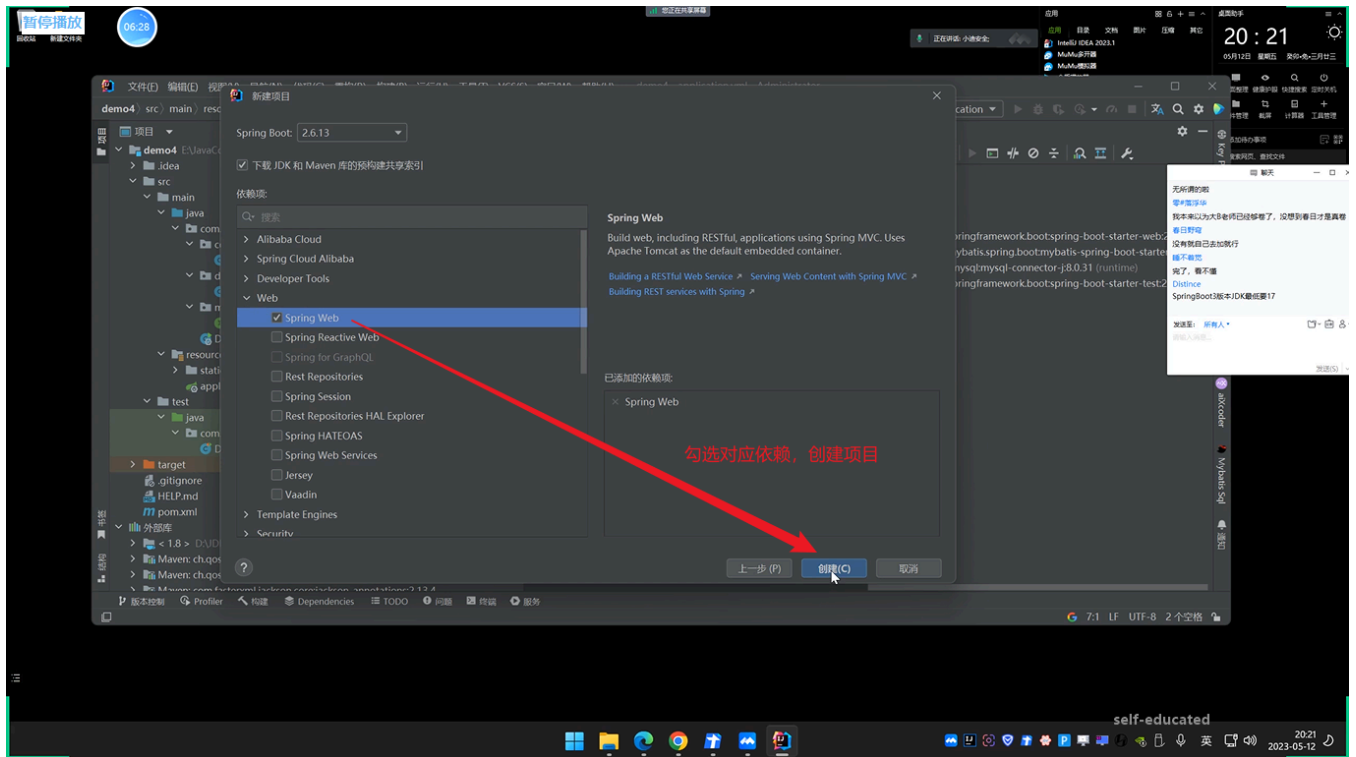
```

11 @RestController
12 public class HelloController {
13     //无参数访问响应
14     @RequestMapping("/xiaodi")
15     public String hello() {
16         return "hello xiaodi";
17     }
18     //无参数指向GET方法访问响应
19     @RequestMapping(value = "/get",method = RequestMethod.GET)
20     public String helloGet(){
21         return "hello get xiadi";
22     }
23     //有参数指向GET方法访问响应
24     @RequestMapping(value = "/getp",method = RequestMethod.GET)
25     public String hellogetp(String name){
26         return "hello get "+name;
27     }
28     //有参数指向POST方法访问响应
29     @RequestMapping(value = "/getpost",method = RequestMethod.POST)
30     public String helloGetParameters(String name){
31         return "hello POST "+name;
32     }
33 }

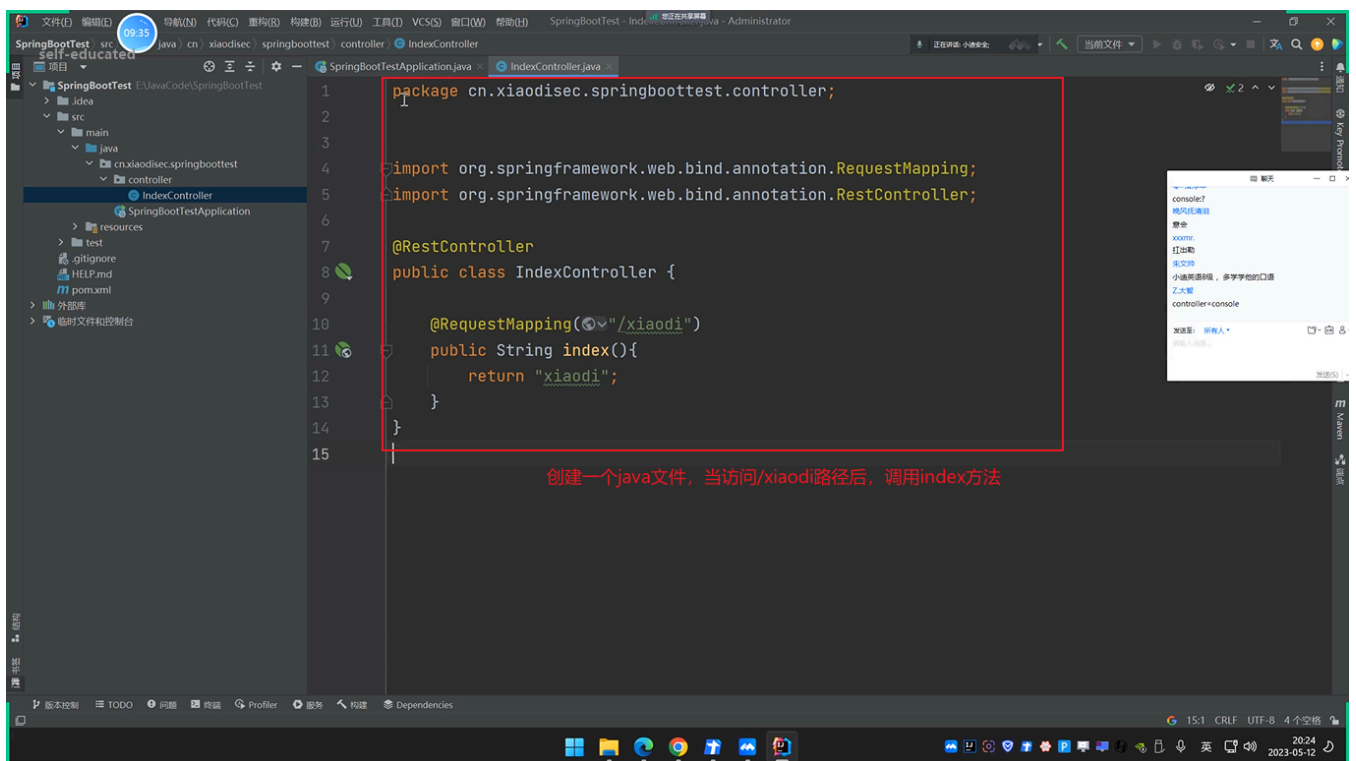
```

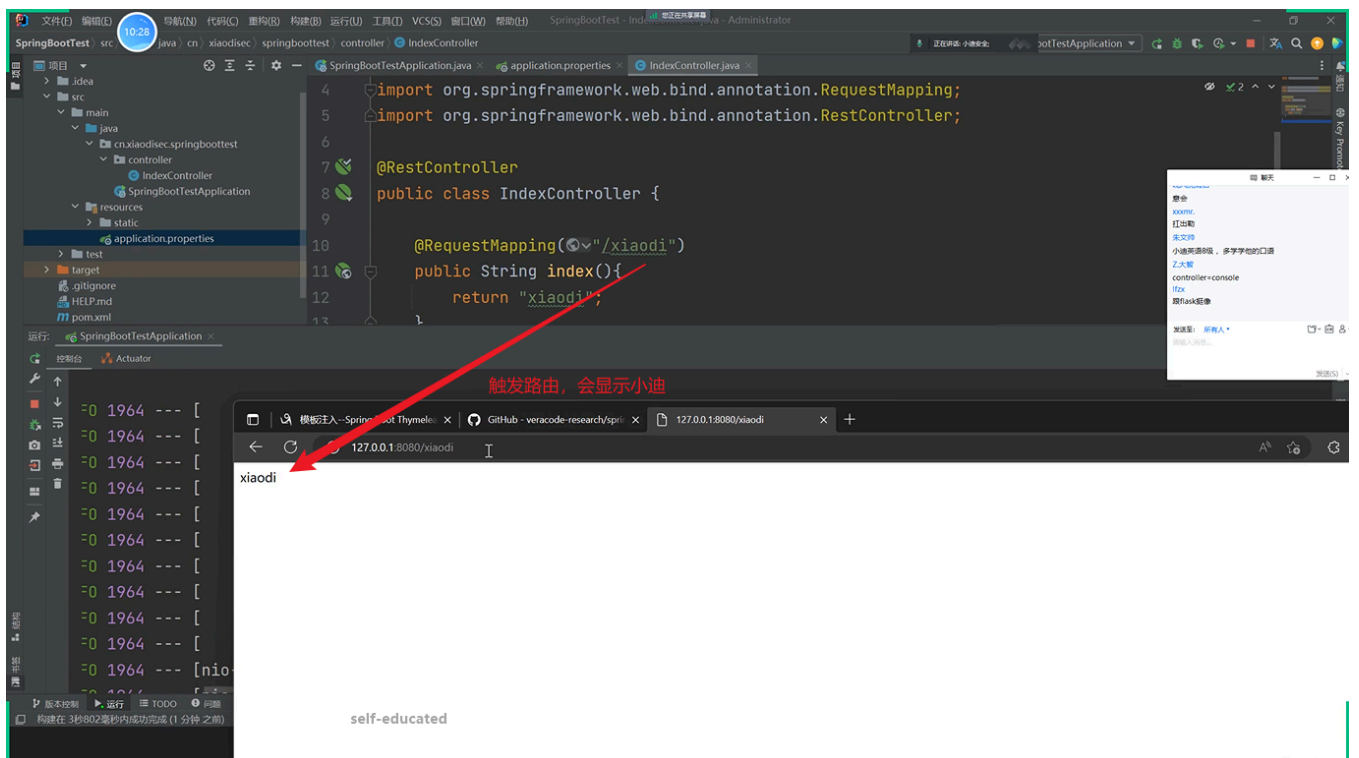
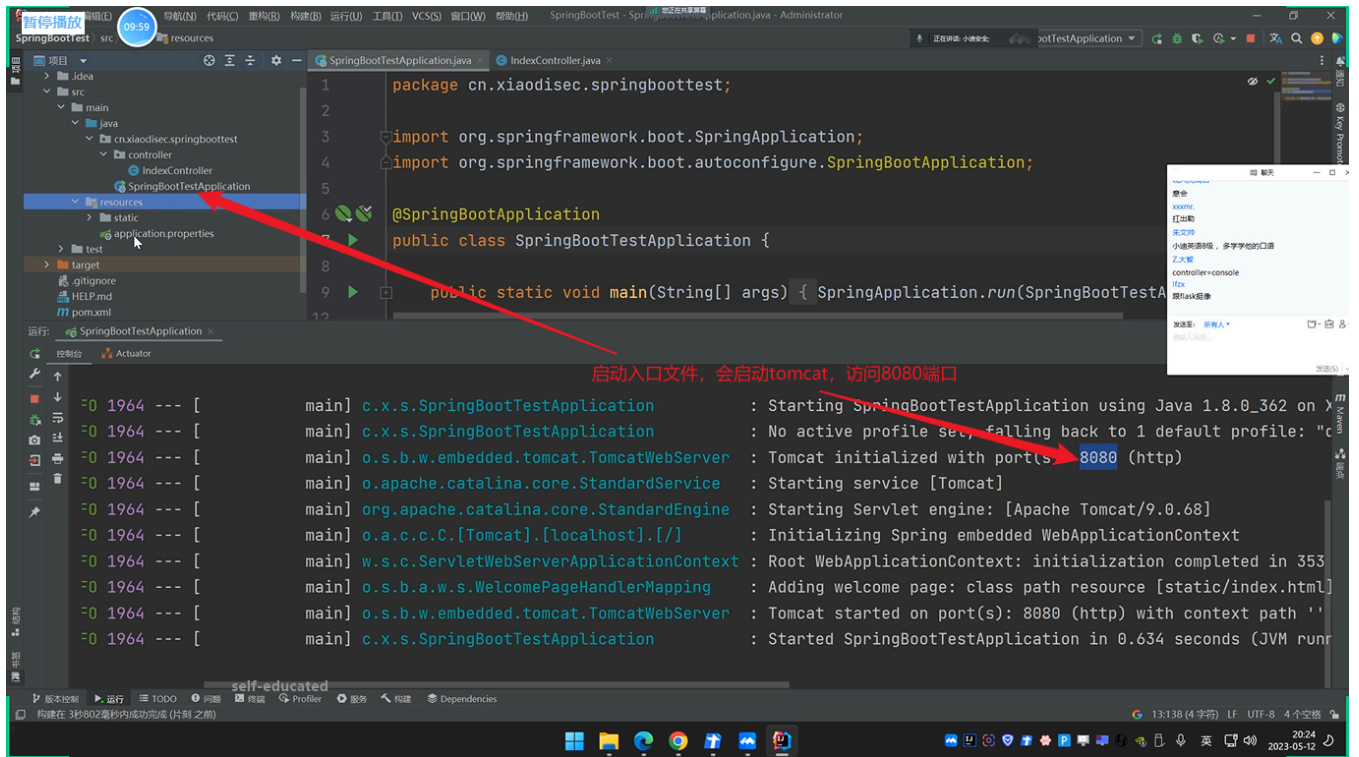
(1) 创建项目：

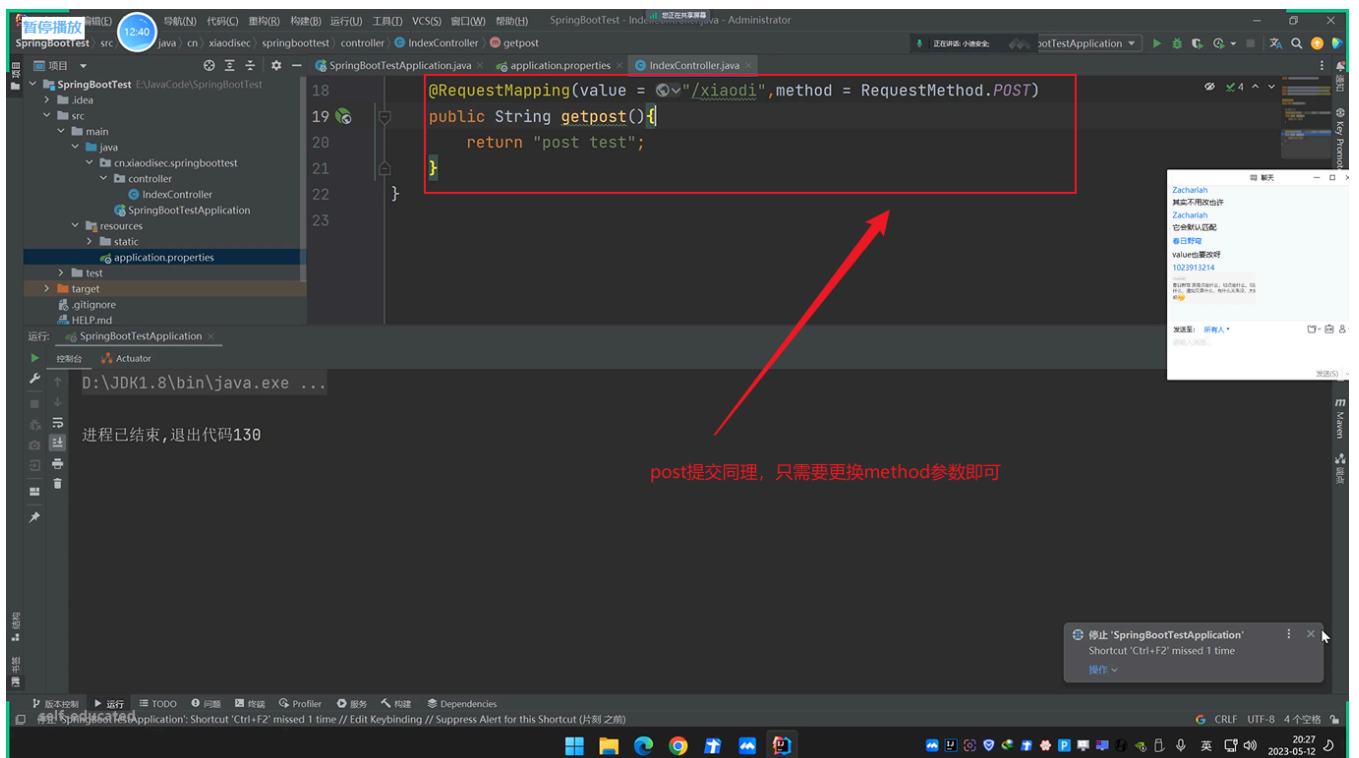
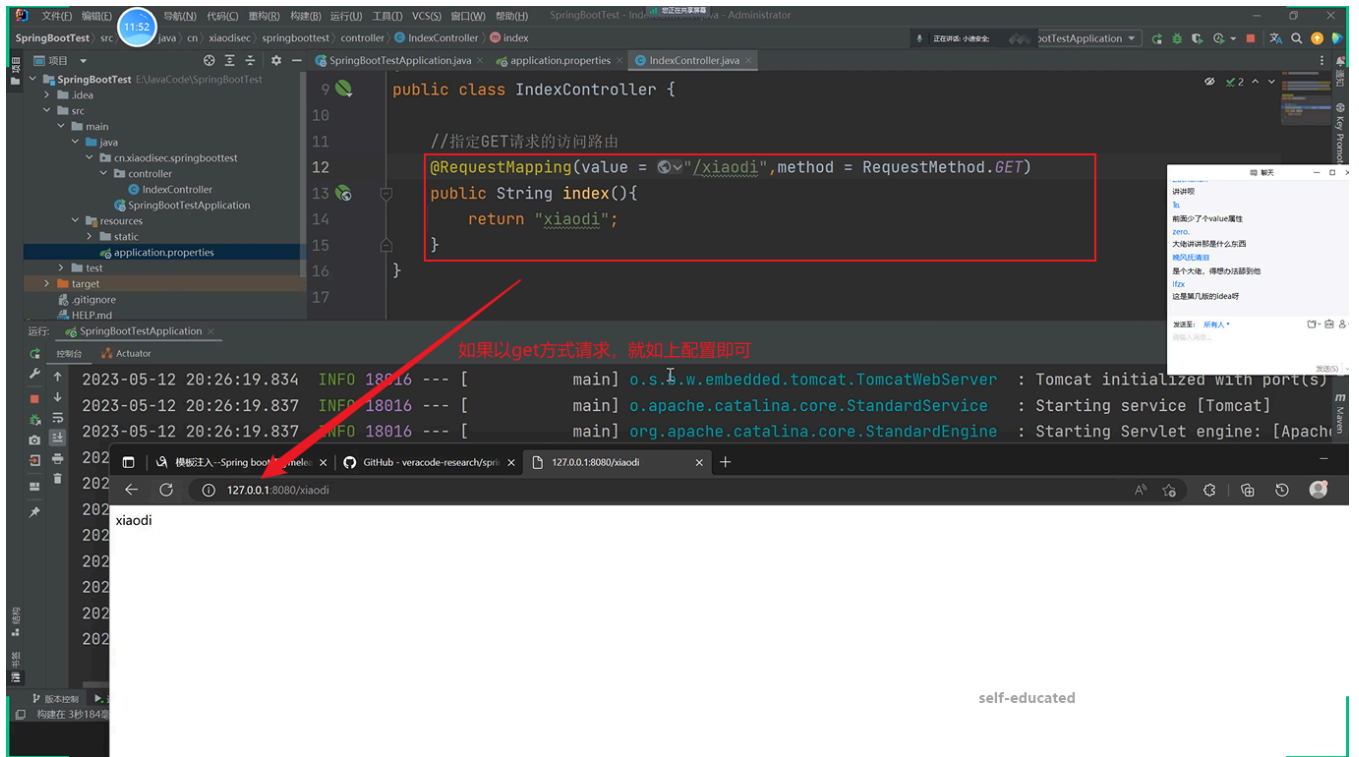




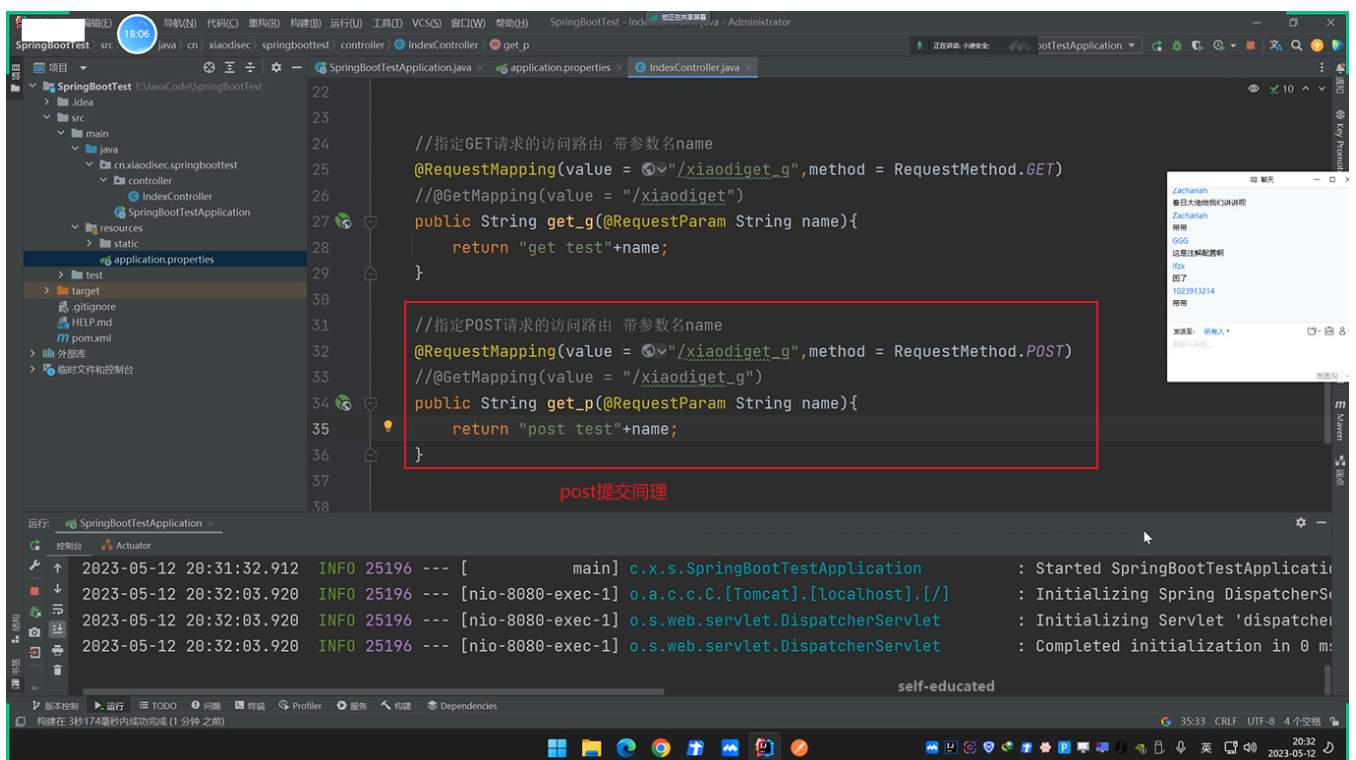
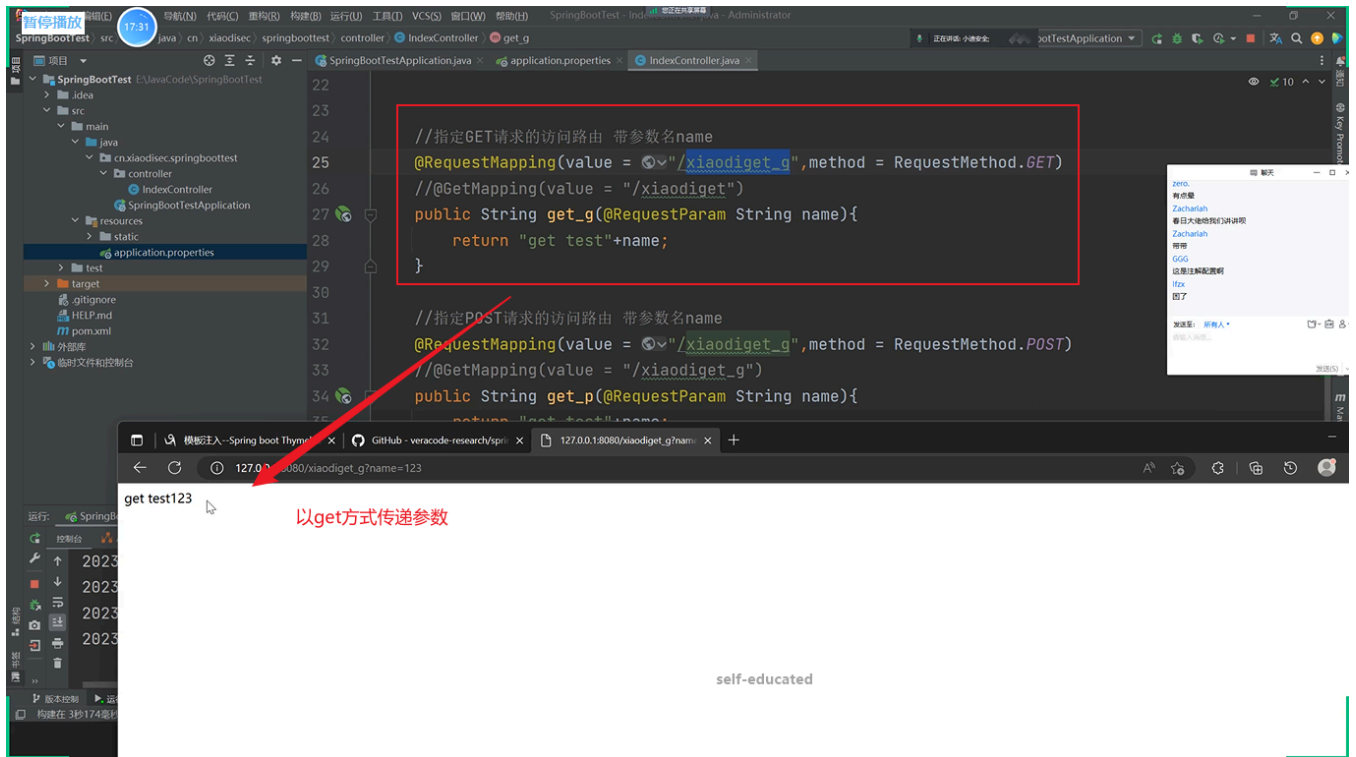
(2) 路由映射:







(3) 参数传递:



(4) 数据响应:

- 1 @RestController @Controller
- 2 @RestController注解相当于@ResponseBody+@Controller合在一起的作用。

2.2 SpringBoot-数据库应用-Mybatis

```
1 0、数据库先创建需操作的数据
2
3 1、项目添加Mybatis&数据库驱动
4 -pom.xml
5 <dependency>
6     <groupId>org.mybatis.spring.boot</groupId>
7     <artifactId>mybatis-spring-boot-starter</artifactId>
8     <version>2.2.2</version>
9 </dependency>
10 <dependency>
11     <groupId>com.mysql</groupId>
12     <artifactId>mysql-connector-j</artifactId>
13     <scope>runtime</scope>
14 </dependency>
15
16 2、项目配置数据库连接信息
17 -application.yml
18 spring:
19     datasource:
20         url: jdbc:mysql://localhost:3306/demo01
21         username: root
22         password: 123456
23         driver-class-name: com.mysql.cj.jdbc.Driver
24
25 3、创建User类用来操作数据库数据
26 -com.example.demo.entity.User
27 set get toString方法
28
29 4、创建Mapper动态接口代理类实现
30 -com.example.demo.mapper.UserMapper
31 @Mapper
32 public interface UserMapper {
33     @Select("select * from admin where id like '%${id}%'")
34     public List<User> findAll(Integer id);
35 }
36
37 5、创建Controller实现web访问调用
38 -com.example.demo.controller.UserController
39 @RestController
40 public class UserController {
41     @Autowired
42     private UserMapper userMapper;
43     @RequestMapping(value = "/getdata", method = RequestMethod.GET)
44     // @ResponseBody
45     public List<User> getdata(Integer id) {
```

```

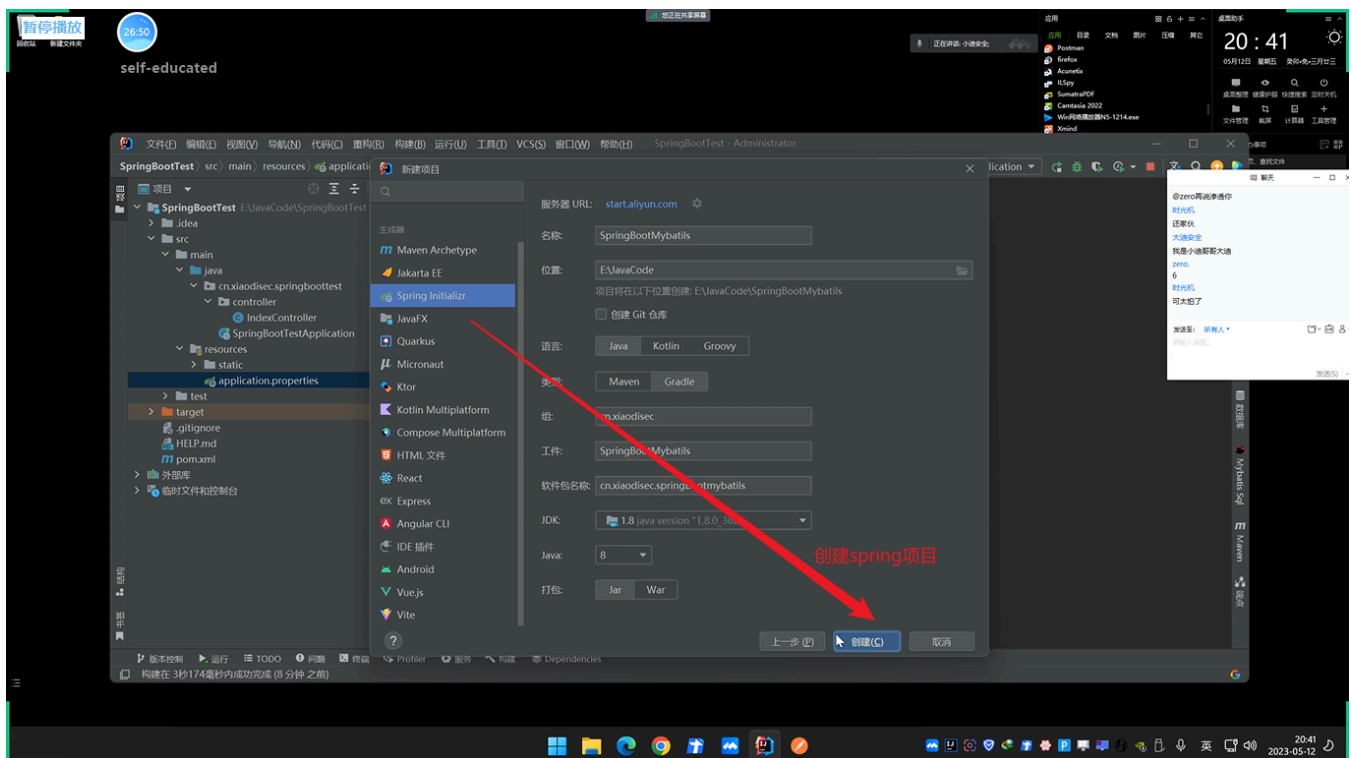
46         List<User> all = userMapper.findAll(id);
47         System.out.println(all);
48         return all;
49     }
50 }

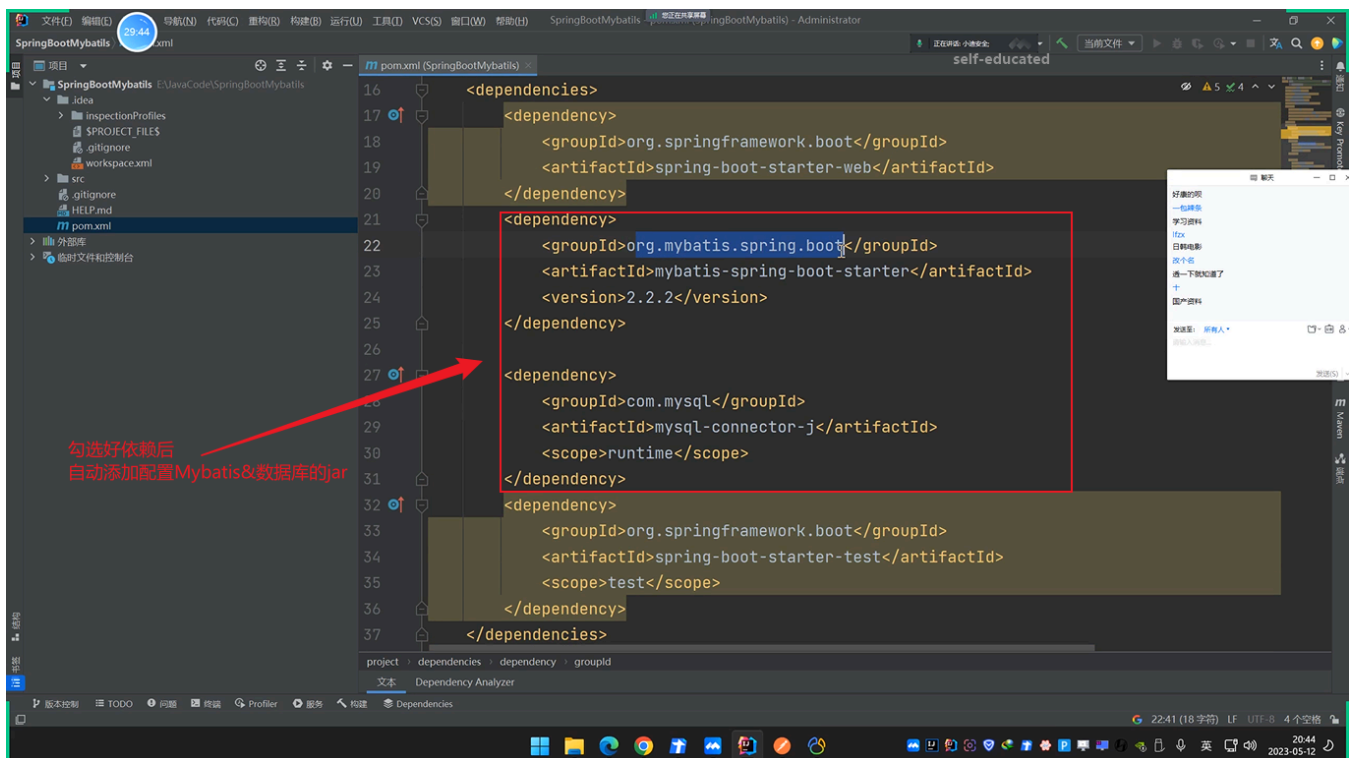
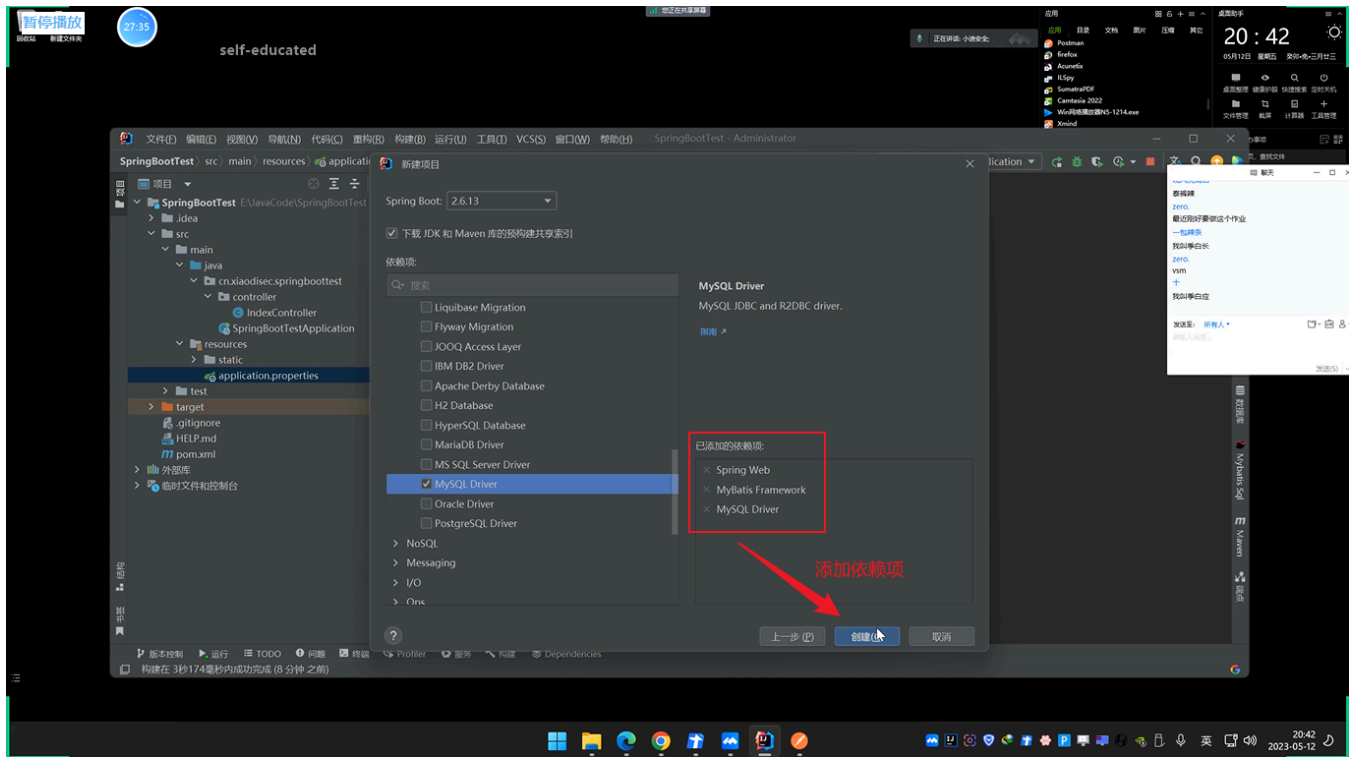
```

(1) 数据库先创建需操作的数据：

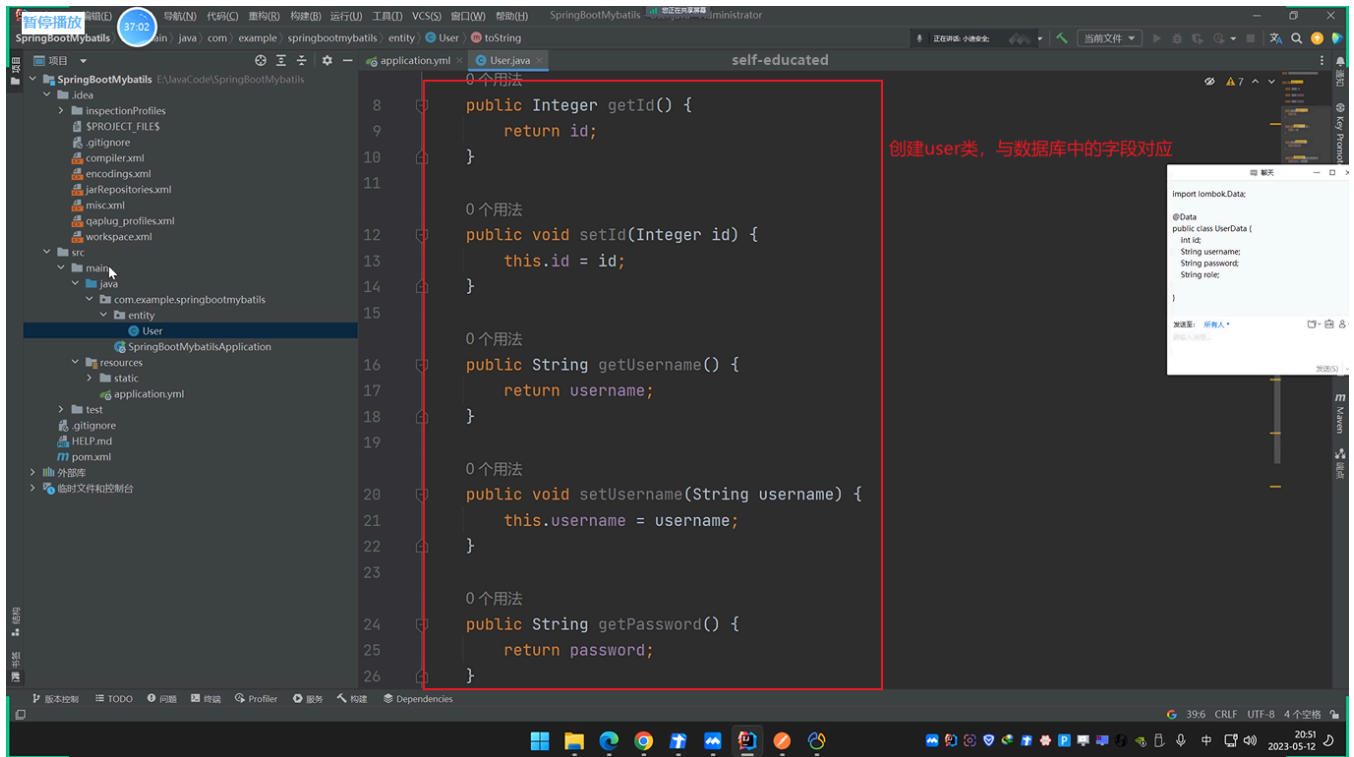
1 前期已经操作，不做演示。

(2) 项目添加Mybatis&数据库驱动：

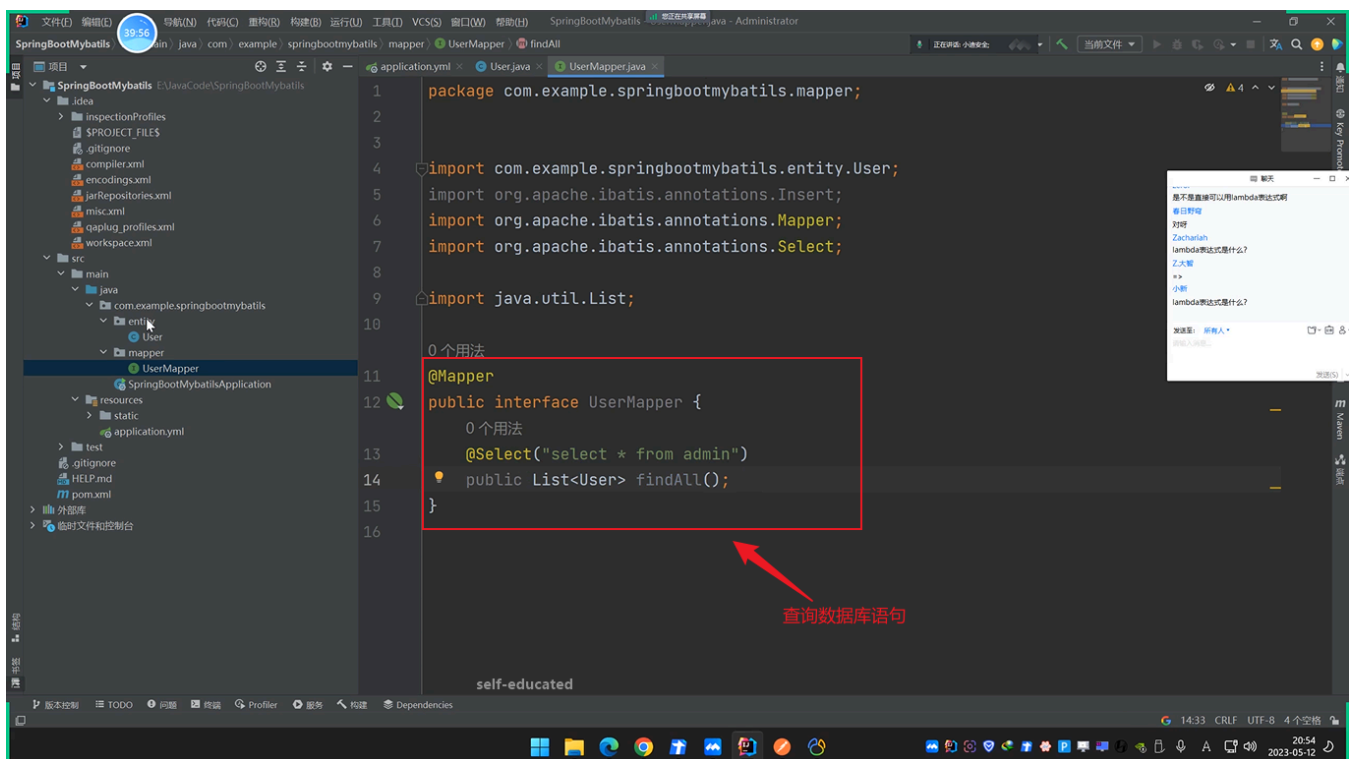




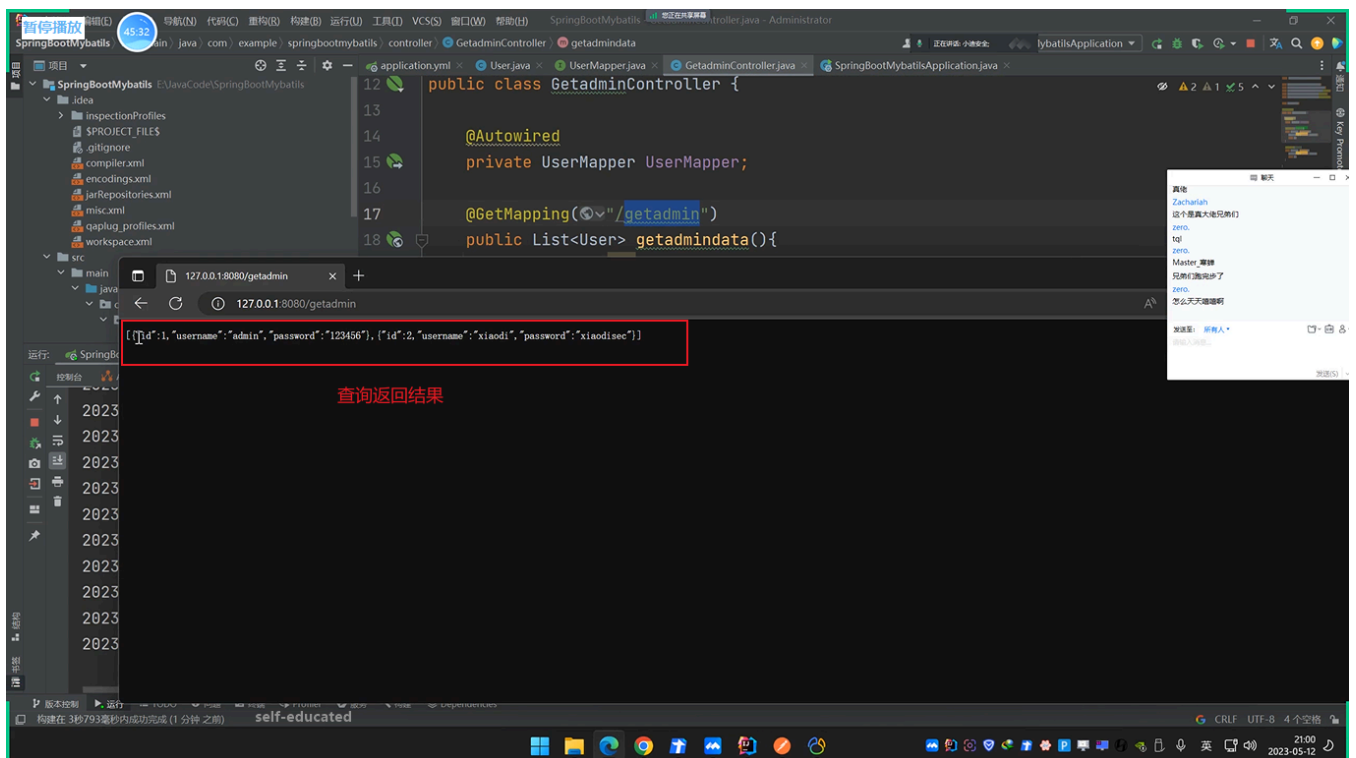
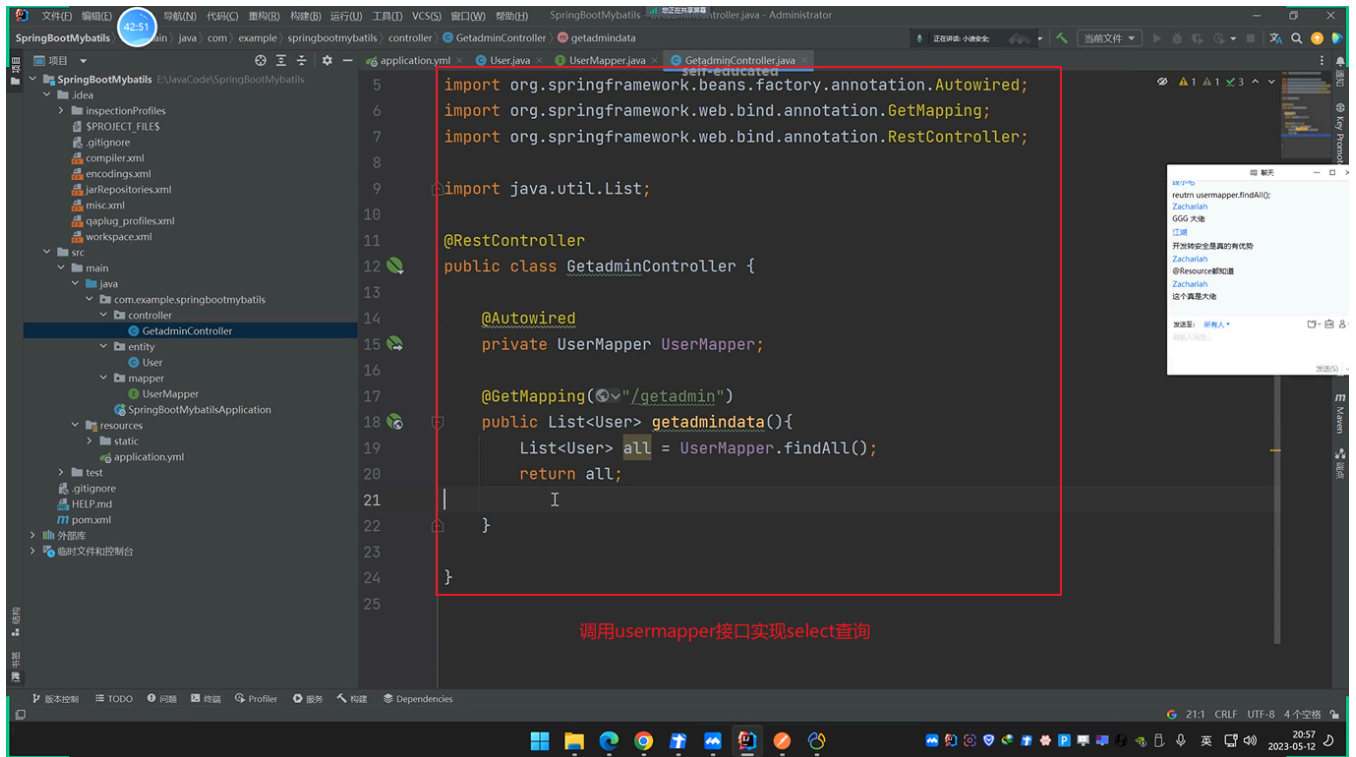
(3) 项目配置数据库连接信息:



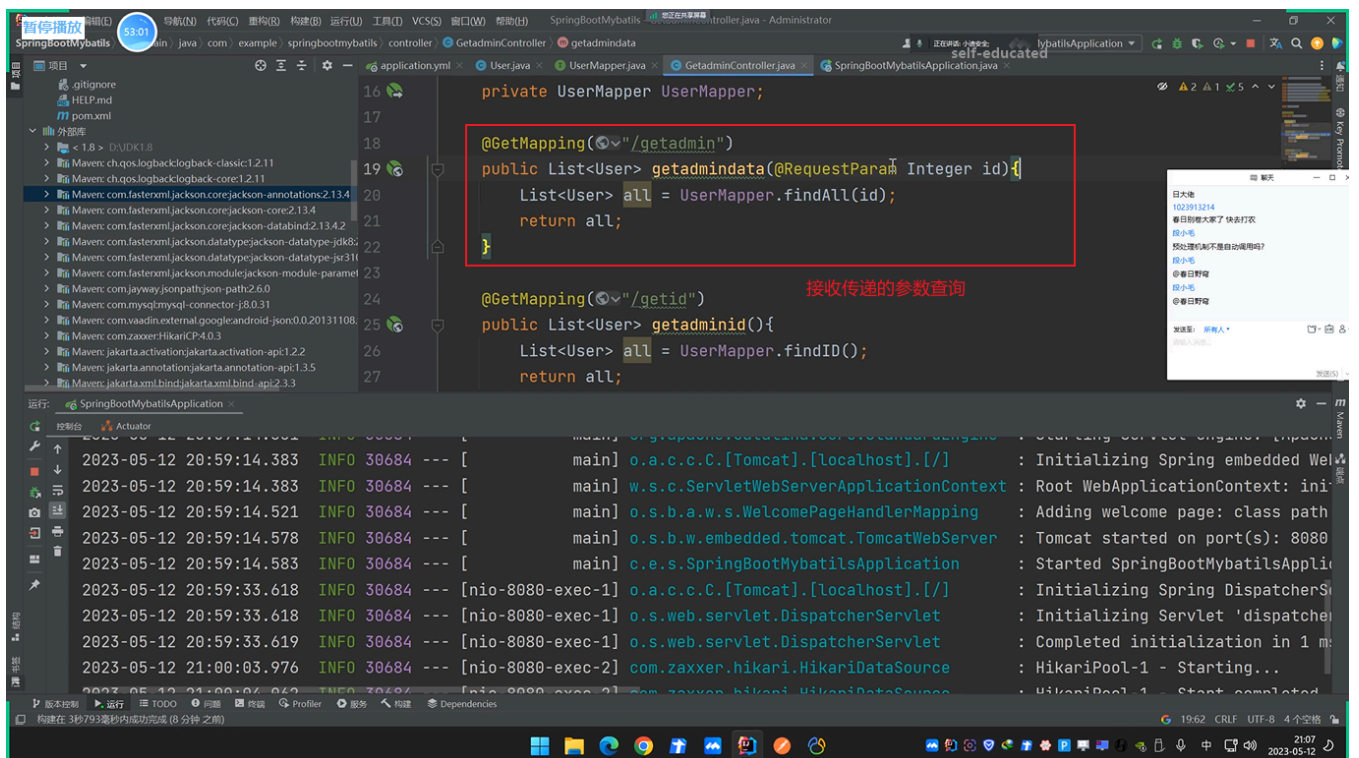
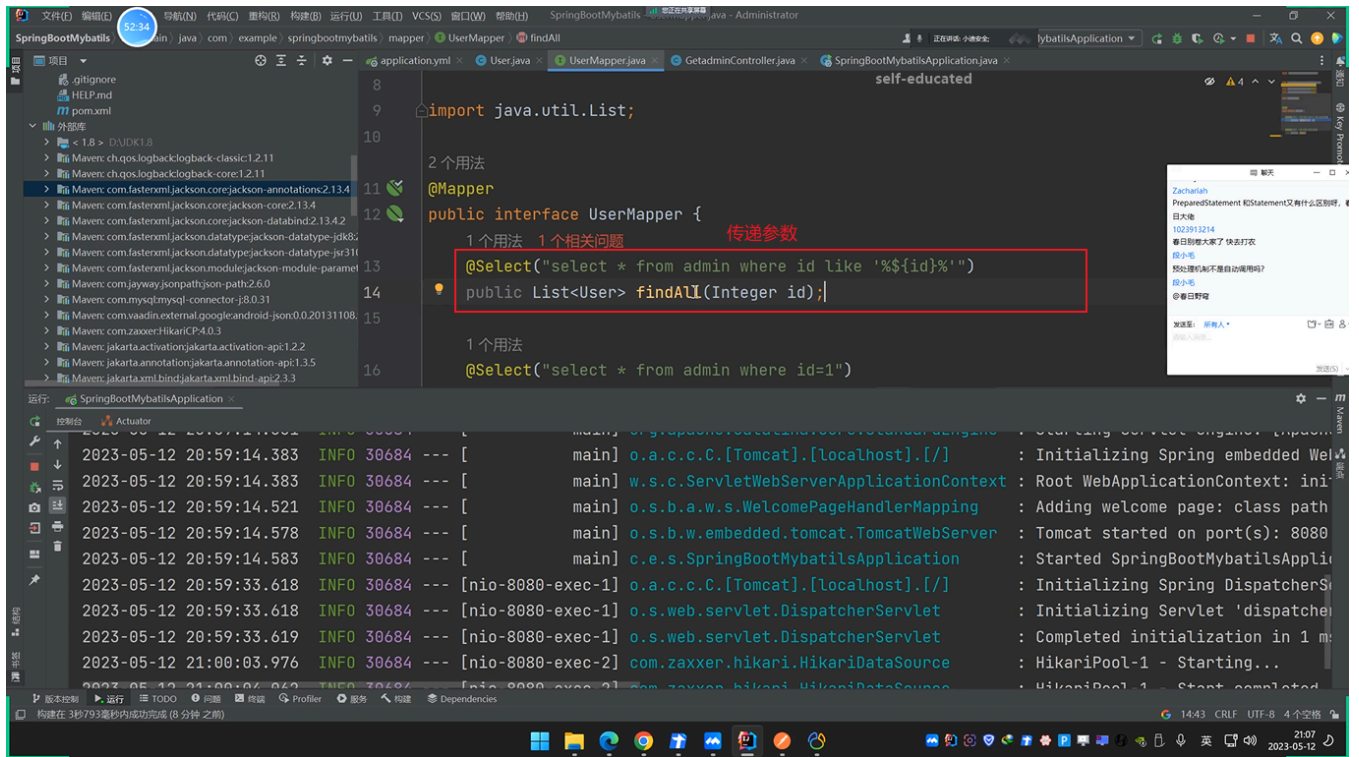
(5) 创建Mapper动态接口代理类实现:

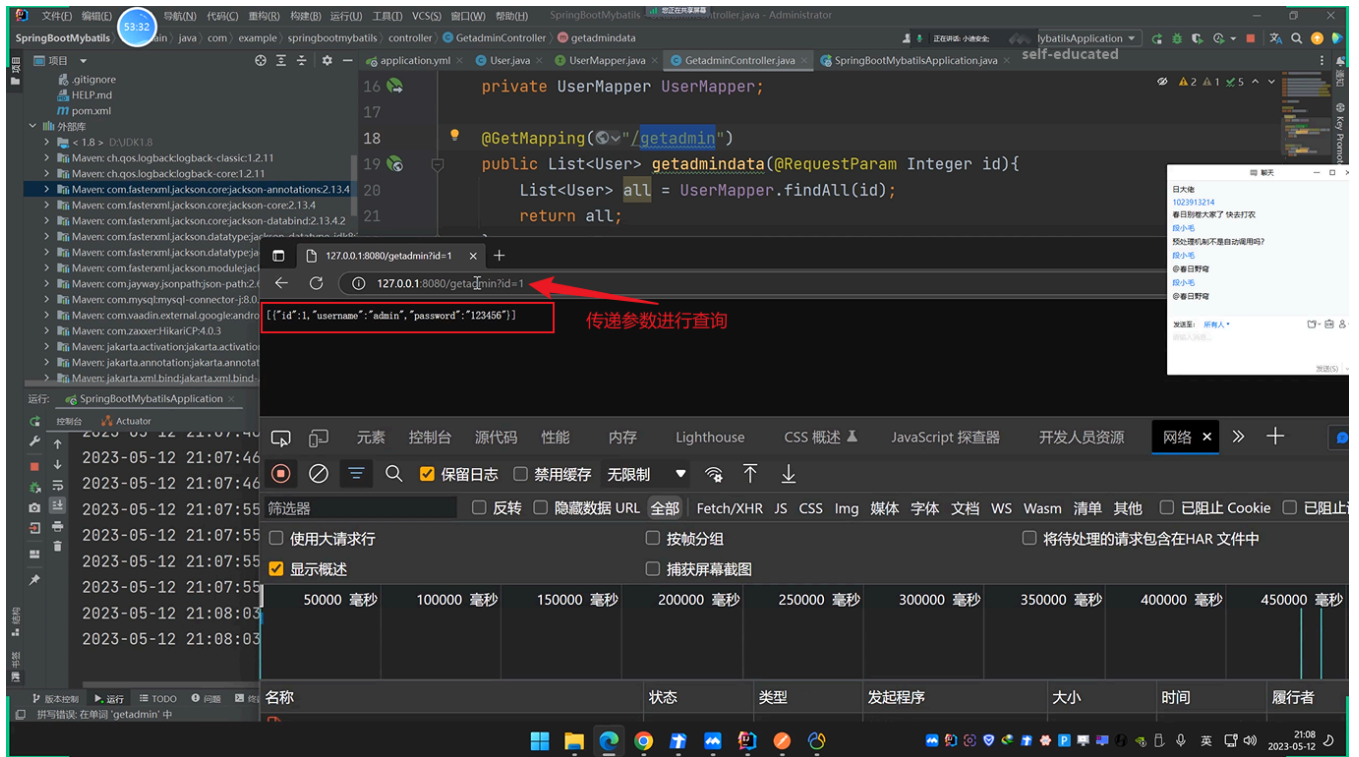


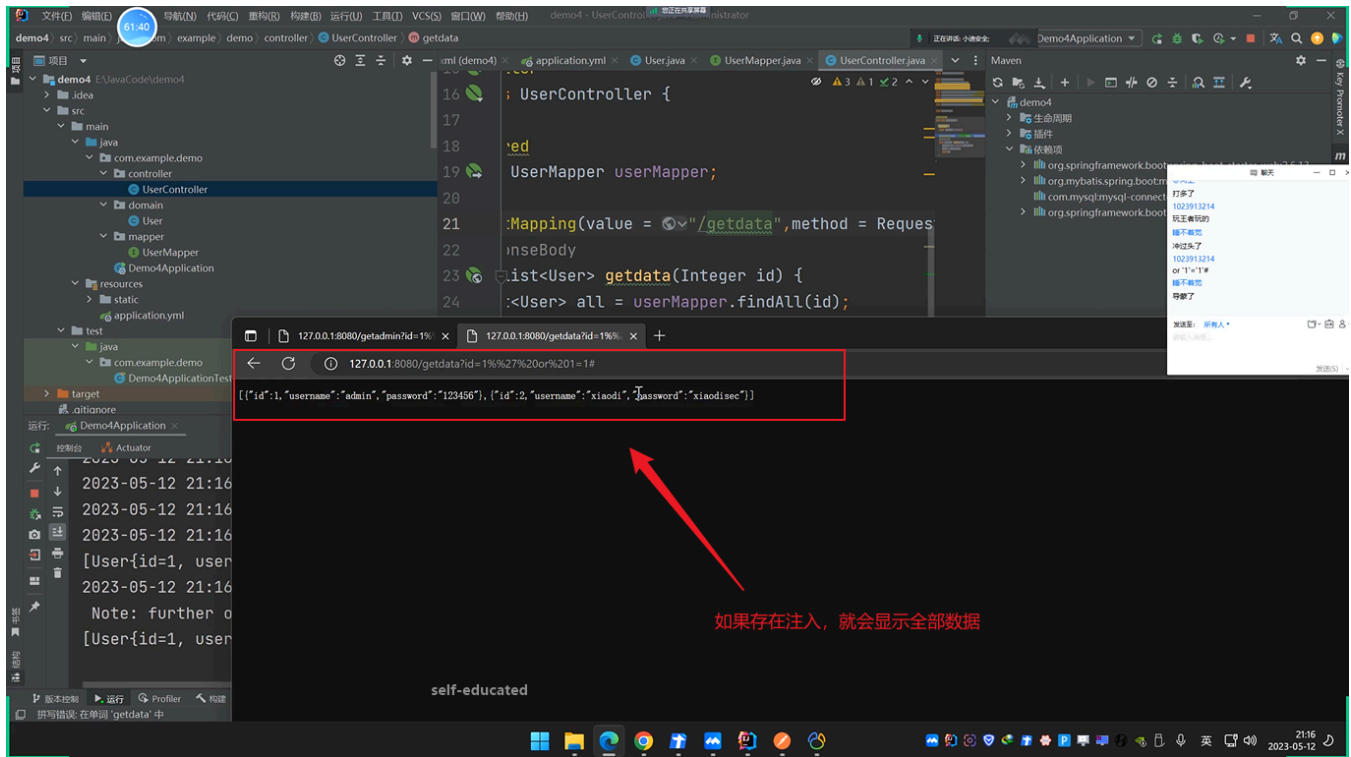
(6) 创建Controller实现Web访问调用:



(7) 参数传递:





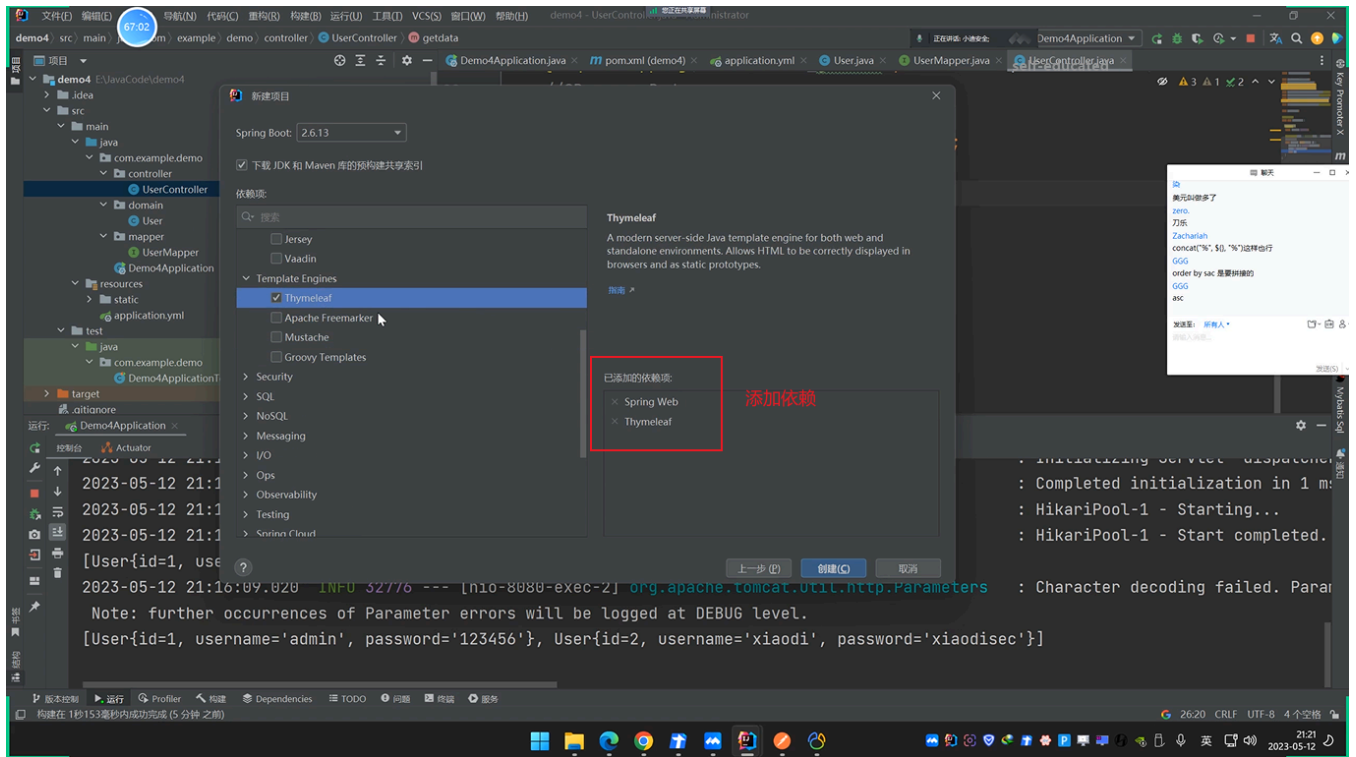


2.3 SpringBoot-模版引擎-Thymeleaf

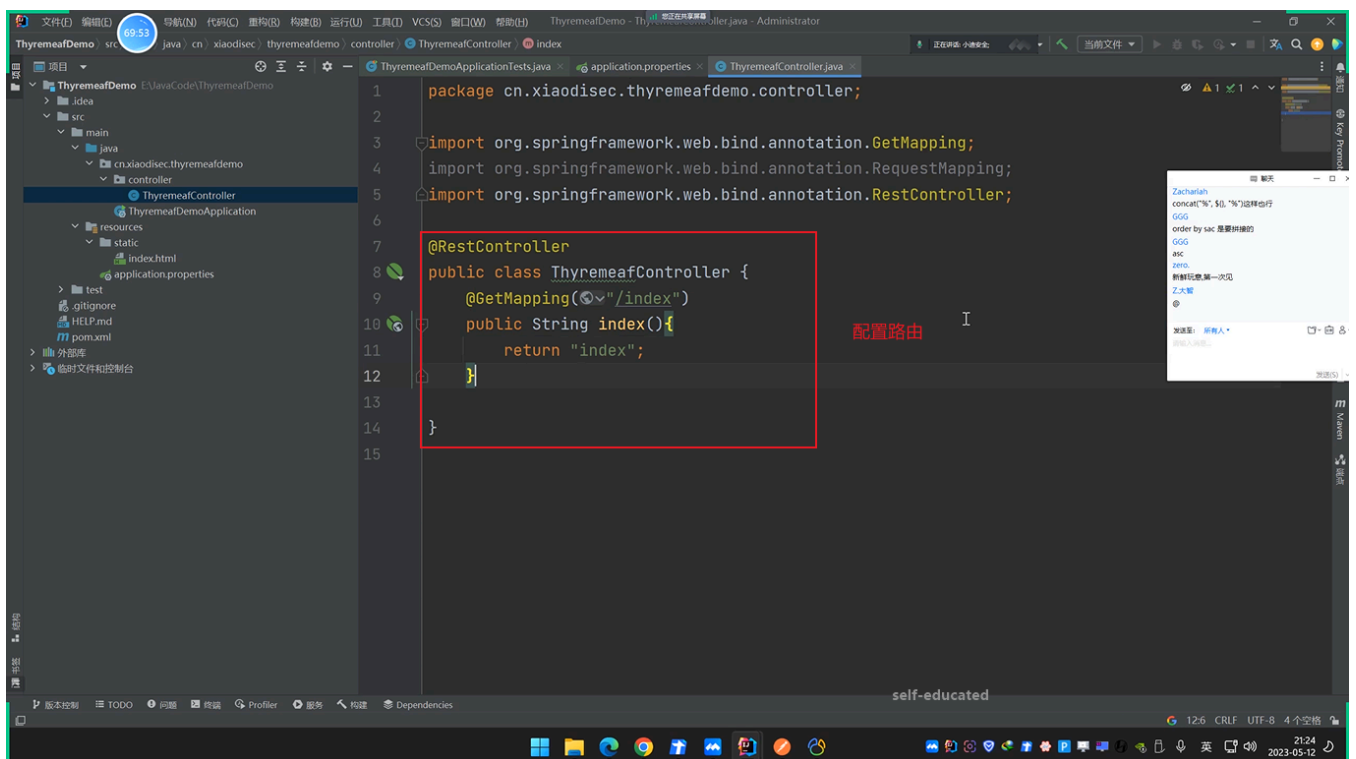
- 1 -不安全的模版版本
- 2 日常开发中：语言切换页面，主题更换等传参导致的SSTI注入安全问题
- 3 漏洞参考：<https://mp.weixin.qq.com/s/NueP4ohS2vSeRCdx4A7yOg>
- 4
- 5 配置application.properties指向模版页面
- 6 `spring.thymeleaf.prefix=classpath:/templates/`
- 7 `spring.thymeleaf.suffix=.html`
- 8
- 9 `@RequestMapping(value = "/")`
- 10 `public String index(Model model) {`
- 11 `model.addAttribute("data","hello xiaodi");`
- 12 `return "index";`
- 13 `}`
- 14 `@GetMapping("/path")`
- 15 `public String thymeleaf(String lang) {`
- 16 `return lang;`
- 17 `}`
- 18
- 19 `<?xml version="1.0" encoding="UTF-8"?>`
- 20 `<project xmlns="http://maven.apache.org/POM/4.0.0"`
- 21 `xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"`
- 22 `xsi:schemaLocation="http://maven.apache.org/POM/4.0.0`
- 23 `http://maven.apache.org/xsd/maven-4.0.0.xsd">`
- 24 `<modelVersion>4.0.0</modelVersion>`
- 25 `<groupId>org.springframework</groupId>`

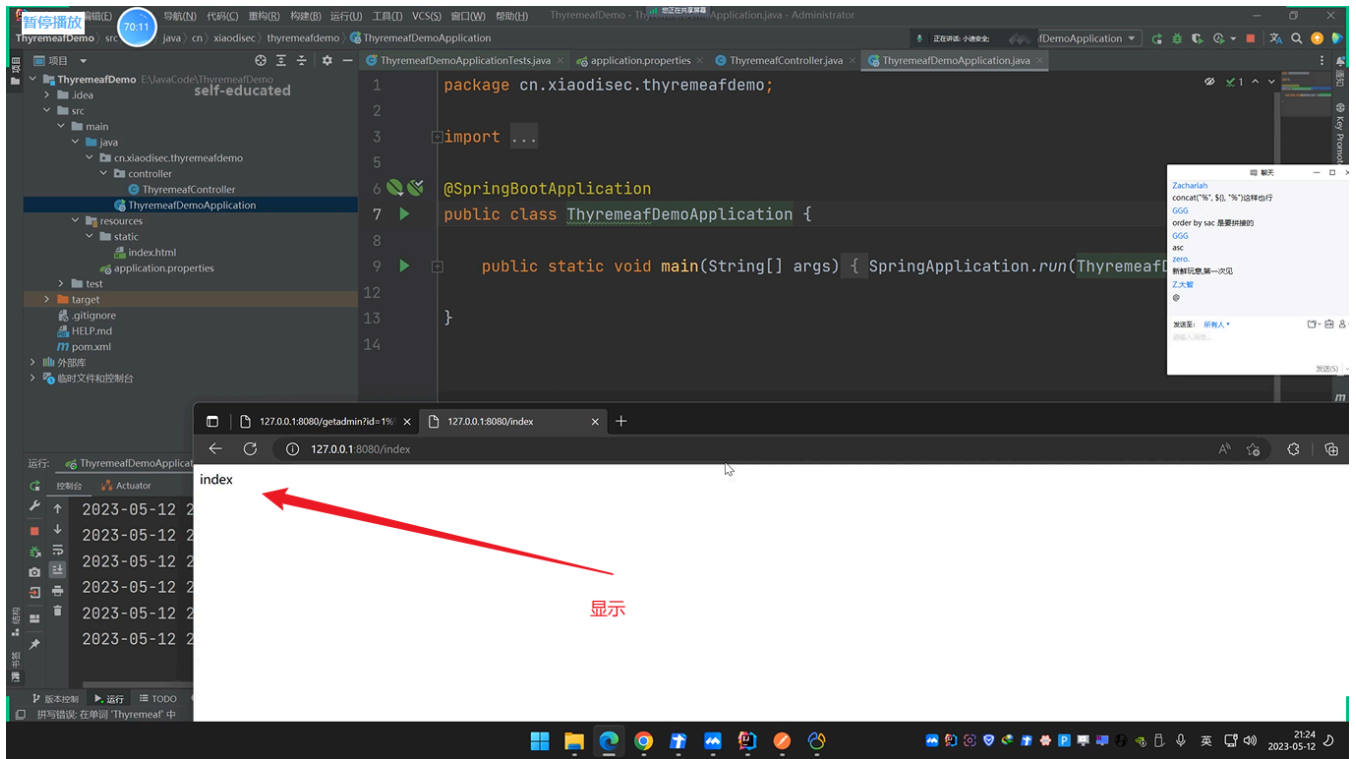
```
25     <artifactId>java-spring-thymeleaf</artifactId>
26     <version>1.0</version>
27
28     <parent>
29         <groupId>org.springframework.boot</groupId>
30         <artifactId>spring-boot-starter-parent</artifactId>
31         <!--latest-->
32         <version>2.2.0.RELEASE</version>
33     </parent>
34
35     <dependencies>
36         <dependency>
37             <groupId>org.springframework.boot</groupId>
38             <artifactId>spring-boot-starter-web</artifactId>
39         </dependency>
40         <dependency>
41             <groupId>org.springframework.boot</groupId>
42             <artifactId>spring-boot-starter-thymeleaf</artifactId>
43         </dependency>
44
45     </dependencies>
46
47     <properties>
48         <java.version>1.8</java.version>
49     </properties>
50
51     <build>
52         <plugins>
53             <plugin>
54                 <groupId>org.springframework.boot</groupId>
55                 <artifactId>spring-boot-maven-plugin</artifactId>
56             </plugin>
57         </plugins>
58     </build>
59 </project>
```

(1) 创建项目，添加项目依赖：

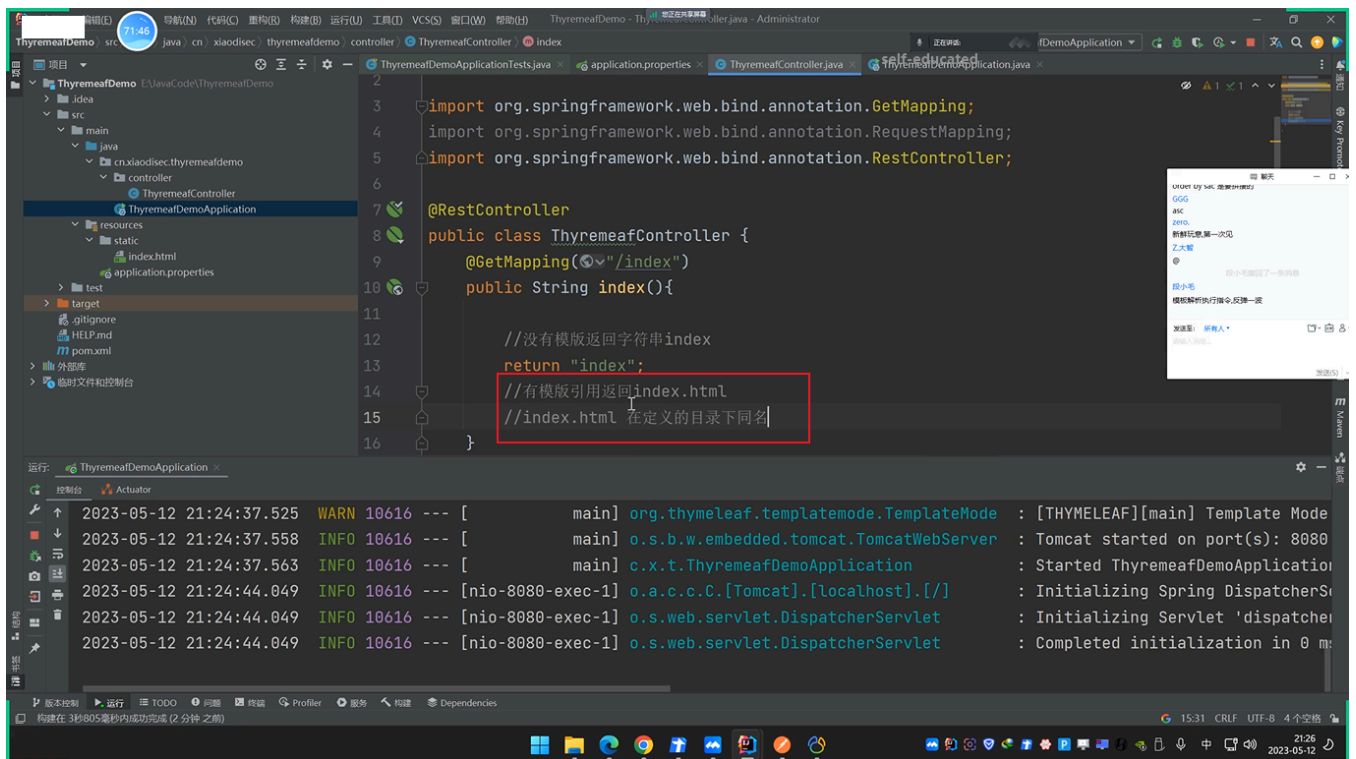


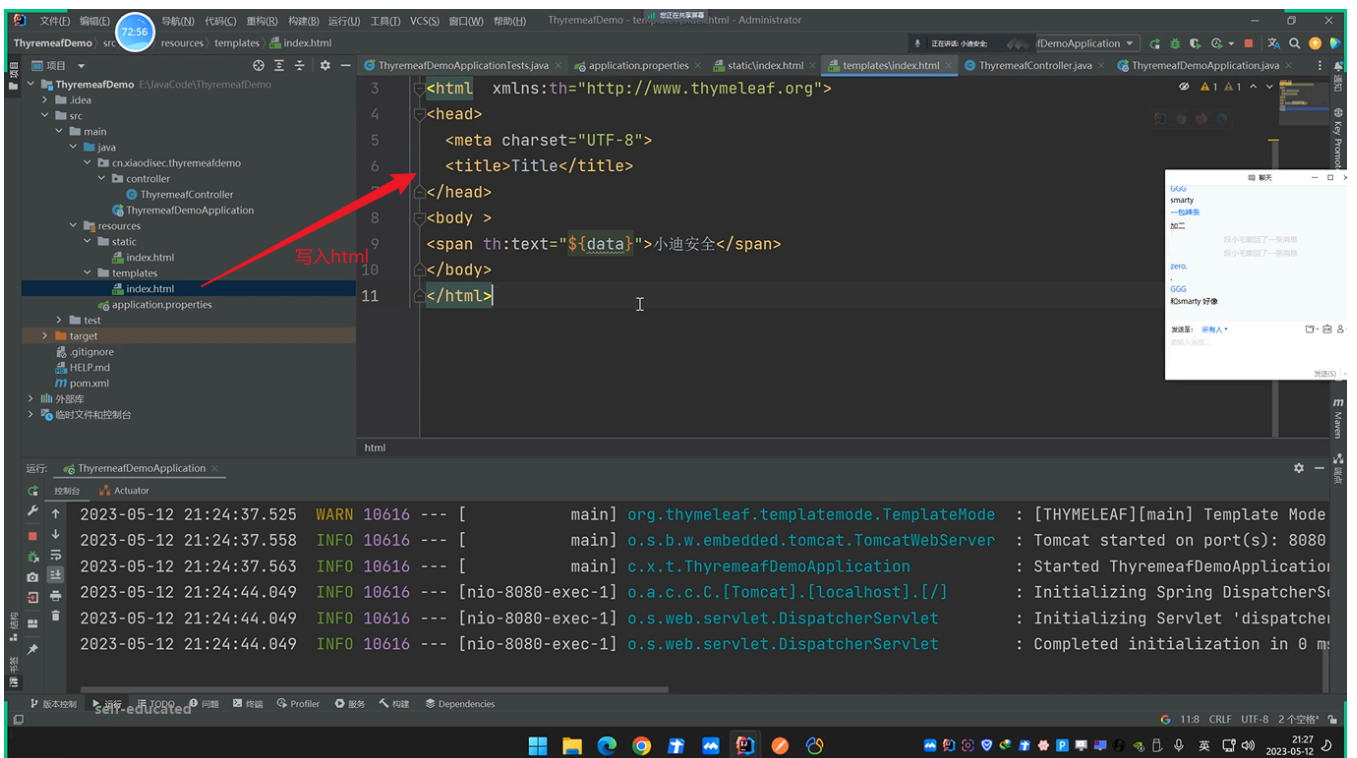
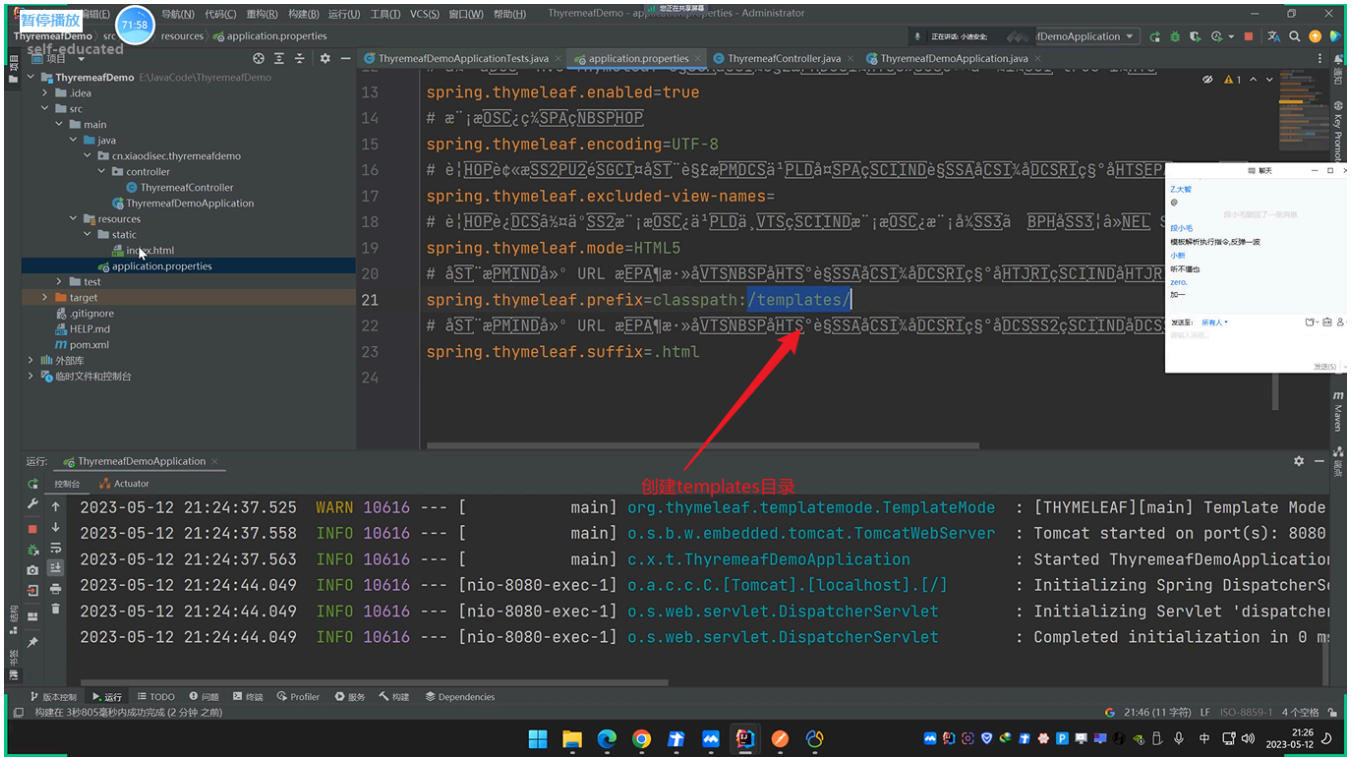
(2) 配置路由访问:

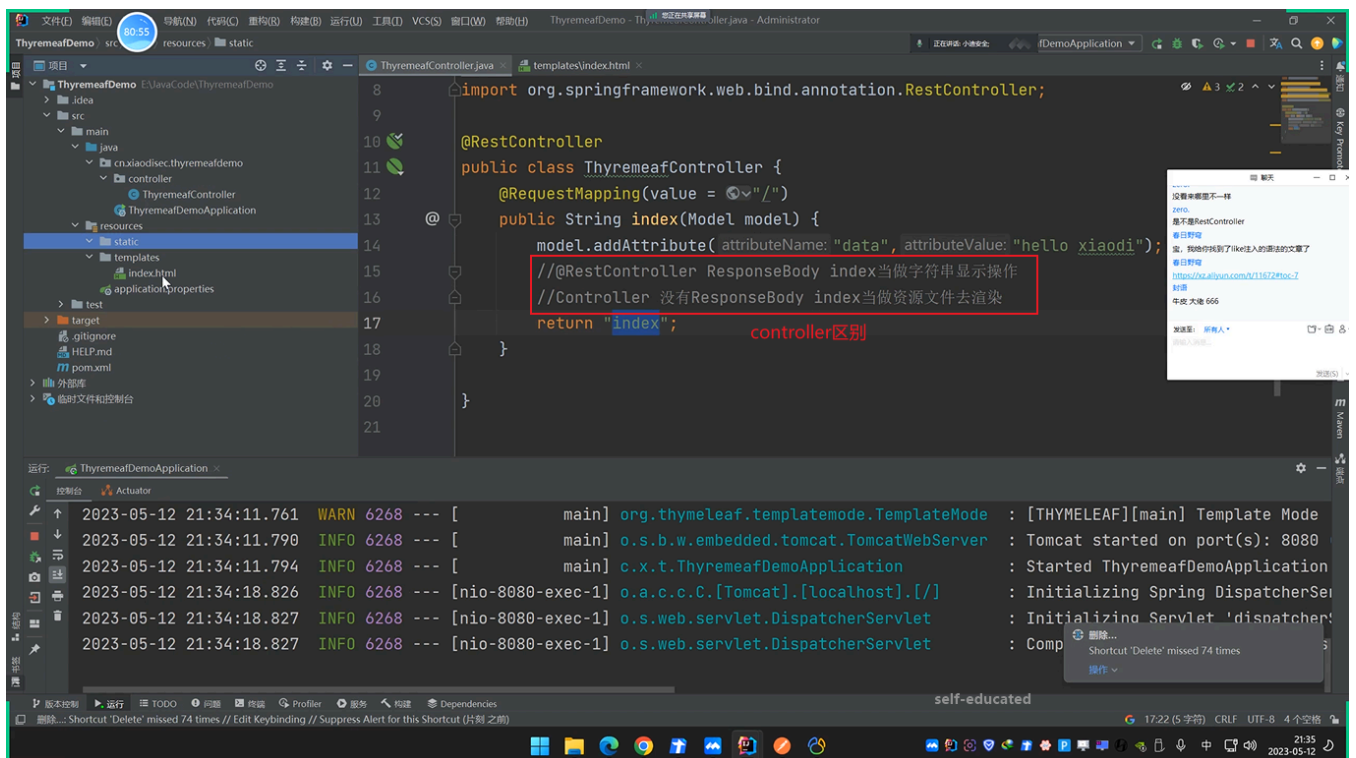
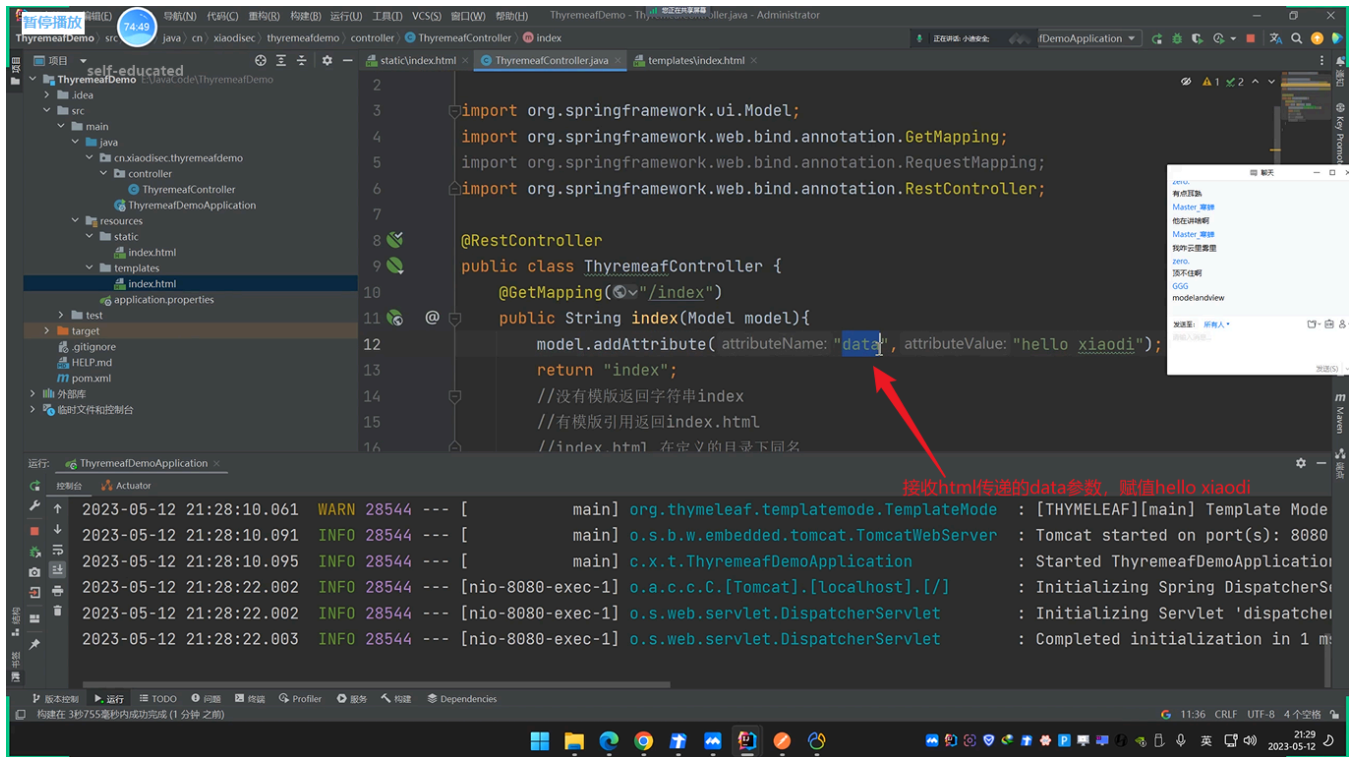


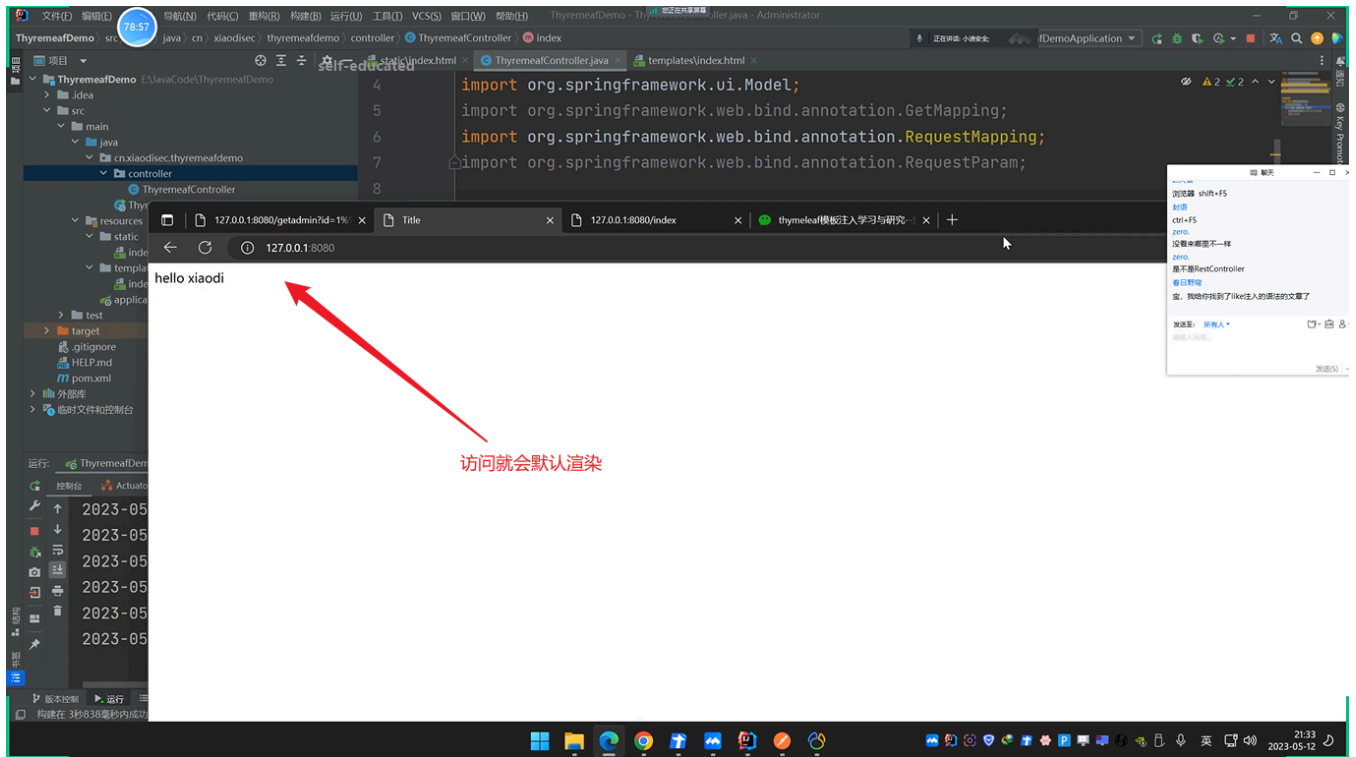


(3) 使用模板进行页面渲染:









(4) 使用模板渲染存在的安全问题:

